

Project - High Level Design

On

HealthCare Content Generator

Course Name: GenAI

Institution Name: Medi-caps University – DataGami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Aparna Sharma	EN24CA5030023
2.	Aditya Bairagi	EN24CA5030009
3.	Shubhanjali Roy	EN24CA5030161
4.	Mimansa Kashyap	EN22IT301054
5.	Harsh Gupta	EN24CA5030057
6.	Sandhya Patidar	EN24CA5030150

Group Name: Group – G-12D12

Project Number: GAI-22

Industry Mentor Name:

University Mentor Name: Ajaj Khan

Academic Year: 2025-2026

Table of Contents

1. Introduction.
 - 1.1.Scope of the document.
 - 1.2.Intended Audience
 - 1.3.System overview.
2. System Design.
 - 2.1.Application Design
 - 2.2.Process Flow.
 - 2.3.Information Flow.
 - 2.4.Components Design
 - 2.5.Key Design Considerations
 - 2.6.API Catalogue.
3. Data Design.
 - 3.1.Data Model
 - 3.2.Data Access Mechanism
 - 3.3.Data Retention Policies
 - 3.4.Data Migration
4. Interfaces
5. State and Session Management
6. Caching
7. Non-Functional Requirements
 - 7.1.Security Aspects
 - 7.2.Performance Aspects
8. References

1. Introduction

The **Healthcare Content Generator** is a Generative AI-based system designed to assist healthcare professionals in creating high-quality, structured medical documents such as patient summaries, clinical reports, and educational materials.

The system leverages **Large Language Models (LLMs)**, **Vector Databases**, and **advanced Prompt Engineering techniques** to transform raw inputs into standardized, professional outputs while maintaining accuracy and consistency.

1.1 Scope of the document

This document provides a detailed description of the Healthcare Content Generator system, including its objectives, architecture, functionalities, and technologies used in the development of the application. It explains how the system utilizes Python, a Vector Database, and a Large Language Model (LLM) API to generate structured healthcare content from user inputs.

The document outlines the system design, workflow, and implementation process, along with the interaction between the frontend and backend components. It

further explains how the system incorporates advanced prompt engineering techniques through a prompt orchestrator to optimize input queries for better output generation. Additionally, the system leverages a Retrieval Augmented Generation (RAG) approach, where relevant healthcare information is retrieved from a vector database and combined with LLM capabilities to generate accurate and context-aware medical content.

The document also highlights the output processing layer, which ensures consistency in formatting, tone, and medical terminology, thereby improving the reliability and quality of generated content. Overall, the system is designed to assist healthcare professionals in creating documentation more efficiently and with reduced manual effort.

1.2 Intended Audience

This document is intended for healthcare professionals, developers, students, and project evaluators who are interested in understanding the design and functionality of the Healthcare Content Generator system. It provides a clear overview of how the application generates structured healthcare content using Generative AI technologies.

The document is particularly useful for developers and researchers seeking to understand the architectural

design and integration of Python, Vector Databases, Large Language Model (LLM) APIs, and prompt engineering techniques in building an AI-powered healthcare content generation system. It also helps project reviewers and examiners gain insight into the system architecture, workflow, key modules, and overall objectives as represented in the project design and diagram.

1.3 System overview

The Healthcare Content Generator is an AI-powered application designed to generate structured and professional healthcare content based on user-provided inputs. The system allows users to enter healthcare-related topics, notes, or keywords, and select the desired output type, such as patient summaries, diagnosis explanations, or treatment-related content.

The system is built using Python and Flask for the backend, along with a user-friendly frontend interface for interaction. It follows a Retrieval Augmented Generation (RAG) approach, where domain-specific healthcare data is stored in a vector database. When a user submits a query, the system retrieves relevant information using similarity search and combines it with the input.

A prompt orchestrator then applies advanced prompt engineering techniques, such as role-based prompting and few-shot learning, to construct optimized prompts. These prompts, along with the retrieved context, are sent to a Large Language Model (LLM) API, which generates accurate and context-aware healthcare content.

The generated output is further processed through a validation and formatting layer to ensure consistency in structure, tone, and medical terminology. The final result is presented to the user as clear, well-organized, and reliable healthcare documentation.

2. System Design

The Healthcare Content Generator system is designed using a modular and scalable architecture that integrates a frontend interface, backend processing layer, prompt engineering module, vector database, and a Large Language Model (LLM) API. The design ensures efficient processing of user inputs and generation of structured, high-quality healthcare content. The system follows a Retrieval Augmented Generation (RAG) approach to enhance the accuracy, relevance, and contextual understanding of the generated outputs.

The system begins with a user interface (frontend) where users enter a healthcare-related topic, notes, or keywords, and select the desired output type. This request is sent to the backend API developed using Python and Flask, which acts as the central processing unit of the system.

Within the backend, a prompt orchestrator processes the input and applies advanced prompt engineering techniques, such as role-based prompting and few-shot learning, to construct optimized prompts. Simultaneously, the input is converted into embeddings using an embedding model, and a similarity search is performed on the vector database (FAISS) to retrieve relevant healthcare information.

The retrieved contextual data is combined with the engineered prompt and forwarded to the LLM API. The LLM then generates structured and context-aware healthcare content, such as patient summaries, symptom descriptions, diagnosis explanations, and treatment recommendations.

The generated output is further passed through a validation and formatting layer, which ensures consistency in structure, tone, and usage of standard medical terminology. Finally, the processed content is sent back to the frontend interface, where it is displayed to the user in a clear and organized format.

This modular design ensures efficient data retrieval, improved accuracy through contextual augmentation, high-quality content generation, and a seamless user experience, while also allowing flexibility for future enhancements and scalability.

2.1 Application Design

The **Healthcare Content Generator** is a layered GenAI-based application that allows healthcare professionals to generate structured medical content from simple inputs. The frontend collects user input and sends it to the backend, where a prompt engineering engine enhances it using templates, context, and medical terminology before passing it to an LLM API for content generation. The output is then refined through formatting, tone validation, and compliance checks to ensure professional and safe medical standards. The system also includes a data layer with a database and vector knowledge base for better accuracy (RAG), along with integrations for EHR systems and export options, while maintaining security and privacy through authentication and compliance mechanisms.

A. Presentation Layer

The **Presentation Layer** of the Healthcare Content Generator consists of a web/mobile user interface designed for healthcare professionals such as doctors and nurses. Built using technologies like React, this

layer allows users to easily input medical topics, patient details, or clinical notes through structured forms. It provides options to select templates (e.g., patient summary) and displays the generated output in a clear, well-formatted manner. The interface ensures a smooth user experience with features like real-time previews, editable outputs, and export options (PDF/DOCX). It communicates with the backend via APIs, sending user inputs and receiving processed content, while maintaining secure access through authentication mechanisms.

B. Application Layer

The **Application Layer** of the Healthcare Content Generator acts as the bridge between the user interface and the core GenAI processing system. It is implemented using backend technologies like FastAPI or Node.js and functions as an API gateway that handles incoming requests from the frontend, processes them, and routes them to the appropriate modules. This layer manages key operations such as request validation, template selection, and coordination with the prompt engineering engine and LLM API. It ensures smooth communication between components, handles business logic, and returns the generated and processed healthcare content back to the presentation layer. Additionally, it supports authentication, error handling, and integration with external services like vector

databases and document export systems, making the overall application efficient and scalable.

C. AI Processing Layer

The **AI Processing Layer** is the core intelligence of the Healthcare Content Generator, responsible for transforming user input into high-quality medical content. It includes the prompt engineering engine, which refines inputs using a template manager (for structured formats like patient summaries), a context builder (to enrich clinical details), and a medical terminology enhancer (to convert simple language into professional terms). The processed prompt is then sent to the LLM API, which generates accurate and structured healthcare content. Additionally, this layer may leverage a vector database for Retrieval-Augmented Generation (RAG) to fetch relevant medical knowledge, improving the accuracy and reliability of the output.

D. Response Formatting Layer

The **Response Formatting Layer** in the Healthcare Content Generator is responsible for refining and structuring the raw output generated by the AI into a professional and usable format. It includes an output formatter that organizes the content into clear sections such as headings, bullet points, and clinical summaries. A tone and consistency validator ensures that the language remains formal, precise, and aligned with medical standards. Additionally, a compliance checker reviews the content to ensure it meets healthcare guidelines.

and avoids unsafe or sensitive information. This layer ensures that the final output is clean, standardized, and ready for direct use or export by healthcare professionals.

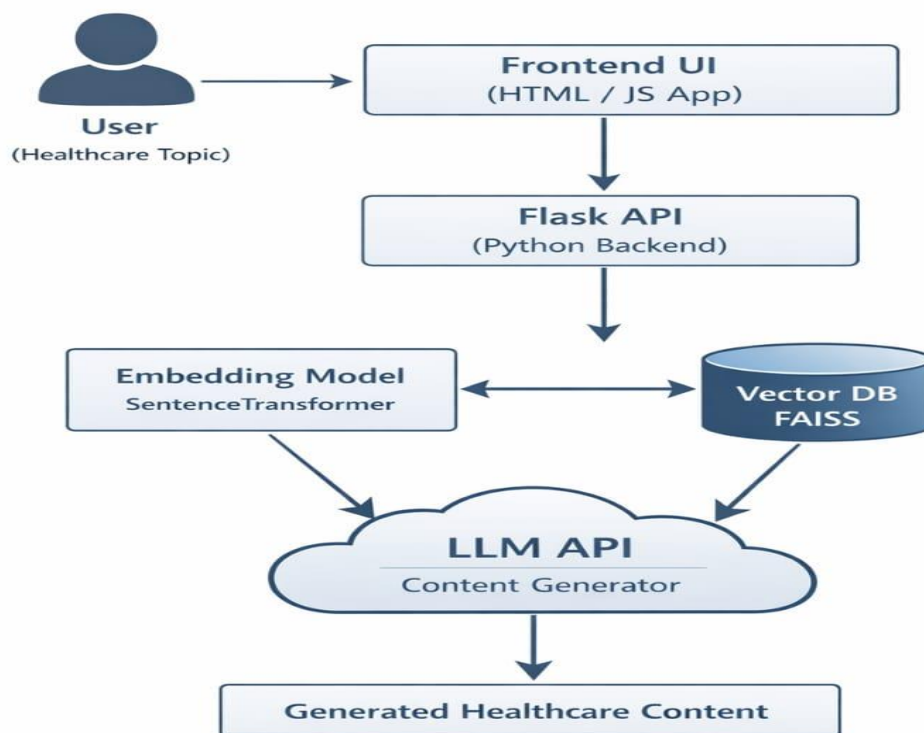
E. Report Generation Layer

The **Report Generation Layer** of the Healthcare Content Generator is responsible for converting the processed and formatted content into final, shareable documents. It takes the structured output from the response formatting layer and generates professional reports such as patient summaries, discharge notes, or clinical documents in formats like PDF and DOCX. This layer ensures proper layout, styling, and document consistency, making the reports ready for real-world medical use. It also supports features like downloading, printing, and integration with external systems such as EHR/EMR platforms, enabling seamless storage and sharing of generated healthcare reports.

2.2 Process Flow

The **process flow** of the Healthcare Content Generator begins when a healthcare professional enters a topic or patient details through the user interface. This input is sent to the application layer, where the backend validates the request and forwards it to the AI processing layer. Here, the prompt engineering

engine selects an appropriate template, enriches the input with context, and enhances it using medical terminology before sending a structured prompt to the LLM API. The model then generates professional healthcare content, which is passed to the response formatting layer for structuring, tone validation, and compliance checks. Finally, the refined content moves to the report generation layer, where it is converted into a downloadable format like PDF or DOCX and presented back to the user for viewing, sharing, or integration with external healthcare systems.



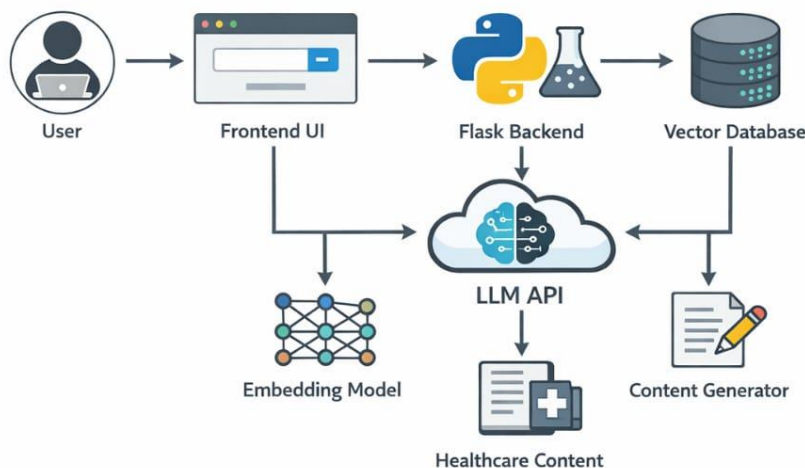
2.3 Information Flow

The information flow begins when the user enters medical data through the frontend, which is sent to the backend via APIs like FastAPI. The input is processed by the prompt engineering module, where it is converted into a structured prompt using templates, context, and medical terminology. This prompt is then sent to the AI model through the OpenAI API, which generates medical content such as patient summaries. The output is passed through post-processing modules for formatting, tone consistency, and compliance checks, then stored in databases like PostgreSQL. Finally, the processed result is displayed on the frontend and can be exported as a PDF or DOCX file or integrated with EHR/EMR systems.

2.4 Component Design

The system is composed of multiple components that work together to ensure efficient healthcare content generation. The User Interface component collects patient data and displays results, while the API controller built using technologies like FastAPI manages communication between frontend and backend. The input is validated and passed to the Prompt Engineering module, which constructs structured prompts using templates, context building, and medical terminology enhancement. This prompt is sent to the AI Processing component via the OpenAI API, which generates professional medical content such as patient summaries. The output then goes through a Post-Processing module that handles

formatting, tone consistency, and compliance validation. The processed data is stored in databases like PostgreSQL and can also integrate with external EHR/EMR systems. Finally, the Report Generation component converts the output into PDF or DOCX format and delivers it to the user through the frontend interface.



2.5 Key Design Considerations

The key design considerations of the Healthcare Content Generator focus on ensuring accuracy, scalability, security, and high-quality output. The system emphasizes strong prompt engineering to generate structured and professional medical content using templates, context building, and terminology enhancement. A modular architecture is used to separate components like UI, backend, AI processing, and

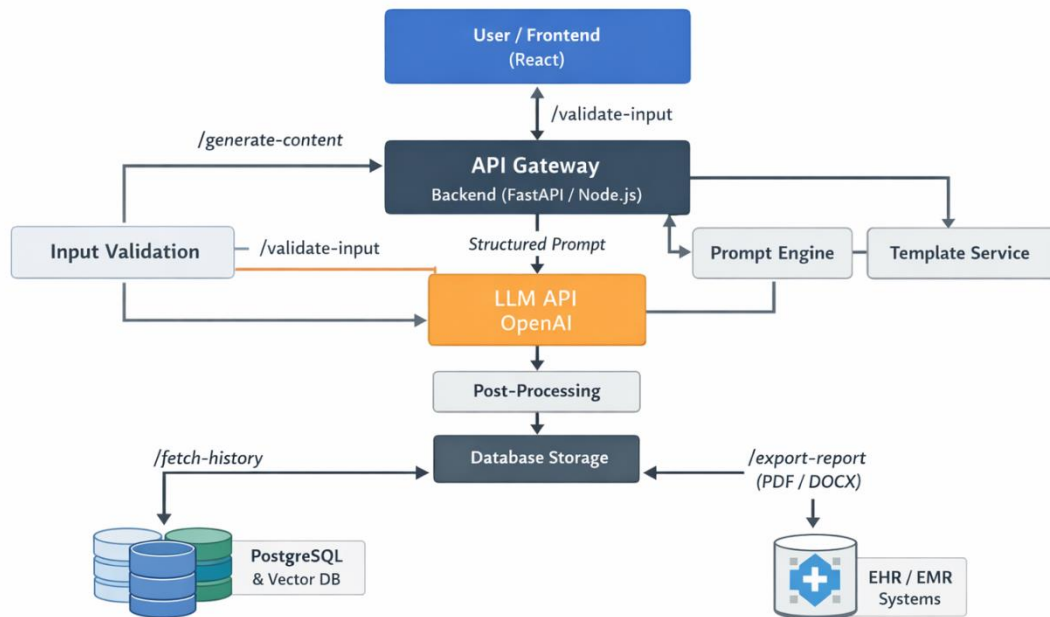
post-processing for better maintainability and scalability. Integration with LLM services such as the OpenAI API is optimized for efficient and reliable content generation. The system also includes post-processing to ensure proper formatting, tone consistency, and compliance with medical standards. Additionally, secure data handling and storage using databases like PostgreSQL are implemented to protect sensitive healthcare information. Finally, the design supports integration with external systems like EHR/EMR and allows export in formats such as PDF or DOCX, making it suitable for real-world healthcare environments.

2.6 API Catalogue

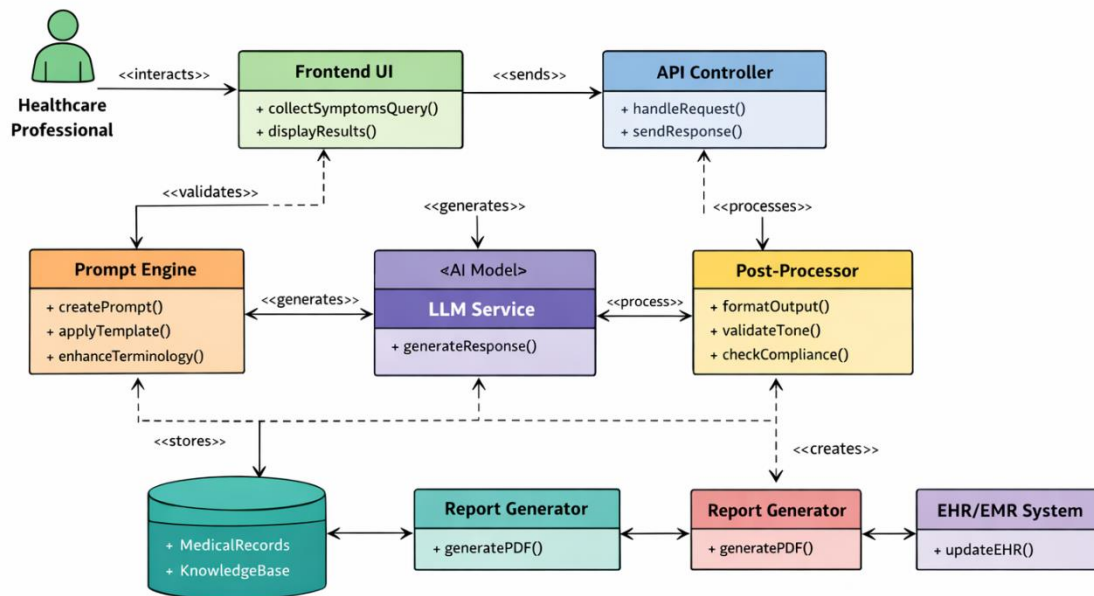
The API Catalogue defines all the APIs used for communication between the frontend, backend, and AI services in the Healthcare Content Generator system. It ensures smooth data flow and modular interaction between components.

The system uses a backend (built with FastAPI or Node.js) to expose REST APIs. The frontend calls these APIs to send patient data and receive generated outputs. The backend then internally communicates with the Prompt Engineering module and external AI services like the OpenAI API.

API Catalogue



UML Diagram



3.Data Design

The data design of the Healthcare Content Generator focuses on efficient storage, retrieval, and management of medical data, templates, and generated content while ensuring security and scalability. The system uses a combination of structured and unstructured data storage. A relational database such as PostgreSQL is used to store structured data including user information, predefined templates, API logs, and generated patient summaries. This ensures data consistency and supports transactional operations.

In addition to the relational database, the system incorporates a Vector Database to store embeddings of medical knowledge and previously generated content. This enables semantic search and context-aware retrieval, improving the quality of prompt construction and AI-generated outputs. The Medical Knowledge Base stores clinical terminology, guidelines, and reference data, which is used by the prompt engineering module to enhance accuracy and domain-specific language.

The data flow is designed such that user input, templates, and contextual data are fetched from the database and passed to the prompt engineering layer, while the generated outputs are stored back for future reference and auditing. Proper indexing and caching mechanisms are used to improve performance. Security measures such as encryption, access control, and compliance with healthcare standards (e.g., HIPAA) are implemented to protect sensitive patient data. Overall, the data design ensures reliability, scalability, and high-quality data handling in the system.

3.1 Data Model

The data model of the Healthcare Content Generator is designed to manage structured medical data, templates, and AI-generated outputs efficiently while supporting scalability and security. The system primarily uses a relational database such as PostgreSQL to store core entities, along with a Vector Database for semantic data handling.

The key entities in the data model include **User**, which stores healthcare professional details; **Patient Data**, which contains symptoms, medical history, and input provided by the user;

and **Template**, which holds predefined formats for generating structured medical documents like patient summaries. The **Prompt** entity represents the structured input created by the prompt engineering module using templates, context, and medical terminology.

The **Generated Content** entity stores the output produced by the AI model, including formatted patient summaries and reports. Additionally, a **Knowledge Base** entity maintains medical terminology, clinical guidelines, and reference data, which can be stored in both relational and vector formats to enable context-aware retrieval. The **Report** entity manages exported files (PDF/DOCX) linked to generated content, while **Audit/Logs** track system activity for monitoring and compliance.

Relationships between these entities ensure smooth data flow: a User creates Patient Data, which is used to generate a Prompt; the Prompt is processed by the AI model to produce Generated Content, which is then formatted into a Report and stored for future access. This data model ensures consistency, traceability, and efficient retrieval of healthcare data while maintaining compliance and security standards.

3.2 Data Access Mechanism

The data access mechanism defines how data is retrieved, processed, and stored across different layers of the system. In this architecture, all data access is handled through the backend API layer, built using technologies like FastAPI or

Node.js, which acts as an intermediary between the frontend, processing modules, and data storage systems.

When a user submits patient data through the frontend, the request is sent to the backend, where it is validated and then used to fetch relevant templates and medical knowledge from the database. The system accesses structured data such as user information, templates, and generated content from a relational database like PostgreSQL. For context-aware retrieval, embeddings are queried from a Vector Database, enabling the prompt engineering module to enrich the input with relevant medical information.

After the AI model generates the output, the backend stores the processed content back into the database for future retrieval, auditing, and reuse. Data access is optimized using indexing, caching, and efficient query handling to reduce latency. Additionally, secure access mechanisms such as authentication, authorization, and encryption are enforced to ensure that sensitive healthcare data is protected and compliant with standards like HIPAA. This layered and controlled data access approach ensures reliability, security, and efficient data flow throughout the system.

3.3 Data Retention Policies

The data retention policy defines how long different types of data are stored, managed, and deleted to ensure compliance, security, and efficient storage utilization. In the Healthcare Content Generator system, sensitive healthcare data such as patient inputs, generated summaries, and reports are retained

only for a defined period based on regulatory and organizational requirements.

User data and generated medical content stored in databases like PostgreSQL are retained for auditing, reuse, and reference purposes, typically for a limited duration unless explicitly required for long-term storage. Templates and system configuration data are retained for longer periods as they are essential for system functionality. Data stored in the Vector Database, such as embeddings of medical knowledge and previous outputs, may be periodically refreshed or pruned to maintain relevance and optimize performance.

The system includes automated data lifecycle management, where outdated or inactive data is archived or securely deleted after a predefined retention period. Backup policies ensure that critical data is stored safely for recovery in case of system failure. Additionally, strict security and compliance measures (such as HIPAA guidelines) are followed, including data encryption, access control, and secure deletion to protect sensitive healthcare information.

Overall, the data retention strategy ensures that the system balances **data availability, compliance, security, and storage efficiency**, while minimizing risks associated with long-term storage of sensitive medical data.

3.4 Data Migration

Data migration in the Healthcare Content Generator refers to the process of transferring existing healthcare data, templates,

and system records from legacy systems or external sources into the new GenAI-based platform. This is especially important when integrating with hospital systems such as EHR/EMR or upgrading from older documentation tools.

The migration process begins with data extraction from source systems, followed by data transformation to match the schema of the target database, such as PostgreSQL. During this stage, data is cleaned, standardized, and validated to ensure accuracy and consistency, particularly for sensitive medical information like patient records and clinical notes. Relevant historical data and templates are also converted into formats compatible with the prompt engineering module.

For advanced features, important textual data may be converted into embeddings and stored in a Vector Database to support semantic search and context-aware AI responses. The transformed data is then loaded into the system’s databases and knowledge base. Throughout the process, strict security measures such as encryption and access control are applied to protect sensitive healthcare data.

The system supports both one-time bulk migration (initial setup) and incremental migration (periodic updates from external systems). Validation and testing are performed after migration to ensure data integrity and system reliability. Overall, the data migration strategy ensures seamless integration, minimal data loss, and smooth transition to the AI-powered healthcare platform.

4. Interfaces

Interfaces in the Healthcare Content Generator define how different components of the system interact and communicate with each other as well as with external systems. The primary interface is the **User Interface (UI)**, built using technologies like React, which allows healthcare professionals to input patient data and view generated outputs. This UI communicates with the backend through REST APIs exposed by the application layer.

The **Backend/API Interface**, implemented using frameworks such as FastAPI or Node.js, acts as the central communication hub. It handles requests from the frontend, interacts with the prompt engineering module, and forwards structured prompts to external AI services like the OpenAI API. It also manages responses and sends processed results back to the UI.

The system also includes **Database Interfaces**, which enable interaction with storage systems like PostgreSQL and Vector Databases for retrieving templates, storing generated content, and accessing medical knowledge. Additionally, **External System Interfaces** are used to integrate with EHR/EMR systems for fetching patient data and exporting generated reports. File export interfaces support formats like PDF and DOCX for downloading reports.

All interfaces are designed to be secure, scalable, and well-defined, using standard protocols (such as HTTP/HTTPS) and

data formats (like JSON), ensuring seamless communication across system components and external integrations.

5. State and Session Management

State and session management in the Healthcare Content Generator ensures that user interactions and data are maintained securely and consistently across multiple requests. When a healthcare professional logs into the system, a session is created and managed by the backend using frameworks like FastAPI or Node.js. Each session is associated with a unique session ID or token, which is used to authenticate and authorize subsequent API requests.

The system follows a **stateless API design**, where each request contains necessary authentication details (such as JWT tokens), while temporary session-related data (like current input, selected templates, or intermediate prompts) is managed either on the client side or through secure caching mechanisms. Persistent state, including user data, templates, and generated content, is stored in databases like PostgreSQL.

Session security is ensured through token expiration, encryption, and role-based access control to protect sensitive healthcare data. Additionally, timeout mechanisms automatically terminate inactive sessions to reduce security risks. This approach ensures scalability, secure user management, and seamless interaction across the system while maintaining compliance with healthcare data protection standards.

6. Caching

Caching in the Healthcare Content Generator is used to improve system performance, reduce latency, and minimize repeated computations, especially during AI processing. The system implements caching at multiple levels to optimize data access and response time. Frequently used data such as templates, medical terminology, and configuration settings are cached in memory to avoid repeated database queries from systems like PostgreSQL.

At the application level (built using FastAPI or Node.js), caching is used to temporarily store recently generated outputs and prompt results. This helps in quickly serving repeated or similar requests without invoking the AI model again, thereby reducing API costs and response time. Additionally, embeddings or frequently accessed context data from the Vector Database can also be cached to speed up prompt construction.

The system uses cache invalidation strategies such as time-to-live (TTL) and event-based updates to ensure that outdated or incorrect data is not served. Sensitive healthcare data is handled carefully, ensuring that only non-sensitive or properly secured data is cached. Overall, caching enhances system efficiency, scalability, and user experience while maintaining data accuracy and security.

7. Non-Functional Requirements

Non-functional requirements define the quality attributes and operational standards of the system to ensure it performs reliably in a healthcare environment. The system is designed for **high performance**, ensuring fast response times for generating medical content by optimizing API calls and integrating efficiently with AI services like the OpenAI API. **Scalability** is achieved through a modular architecture that allows the system to handle multiple concurrent users and increasing workloads without performance degradation.

Security and privacy are critical, as the system handles sensitive healthcare data. Strong authentication, authorization, encryption, and compliance with standards such as HIPAA are implemented, along with secure storage using databases like PostgreSQL. The system also ensures **high availability and reliability**, minimizing downtime and supporting continuous access for healthcare professionals.

Additionally, **usability** is emphasized through a simple and intuitive user interface built with React, enabling easy interaction for non-technical users. **Maintainability** is supported by a modular design, making it easier to update components like prompt templates or AI models. Finally, **compliance and accuracy** are ensured through post-processing checks, guaranteeing that generated medical content is consistent, properly formatted, and adheres to healthcare standards.

7.1 Security Aspects

Security is a critical aspect of the Healthcare Content Generator as it handles sensitive patient and medical data. The system implements strong **authentication and authorization** mechanisms to ensure that only verified healthcare professionals can access the platform. Secure session management and token-based authentication are handled through backend frameworks like FastAPI or Node.js.

All data transmitted between the frontend and backend is protected using **HTTPS encryption**, preventing unauthorized access during communication. Sensitive data such as patient information and generated medical reports are securely stored in databases like PostgreSQL with encryption and strict access control policies. Role-based access control (RBAC) ensures that users can only access data relevant to their role.

The system also incorporates **data privacy and compliance measures**, following healthcare standards such as HIPAA to protect patient confidentiality. Input validation and sanitization are implemented to prevent common security threats like injection attacks. Additionally, audit logs are maintained to track system activities and detect suspicious behavior.

Overall, the security design ensures **data protection, secure access, compliance, and system integrity**, making the application safe for real-world healthcare use.

7.2 Performance Aspects

Performance is a key consideration in the Healthcare Content Generator to ensure fast and efficient generation of medical content. The system is designed to provide **low response time** by optimizing API communication between the frontend and backend using frameworks like FastAPI or Node.js. Efficient integration with AI services such as the OpenAI API ensures quick processing of prompts and generation of results.

To improve speed and reduce load, the system uses **caching mechanisms** for frequently accessed data such as templates, medical terminology, and previously generated outputs, minimizing repeated computations and database queries to systems like PostgreSQL. The architecture also supports **scalability**, allowing the system to handle multiple concurrent users through optimized request handling and modular design.

Additionally, **asynchronous processing** is used for time-consuming tasks such as report generation and AI responses, preventing delays in user interaction. Performance is further enhanced through efficient database indexing, load management, and optimized data retrieval from vector databases. Overall, the system ensures **high throughput, low latency, and smooth user experience**, making it suitable for real-time healthcare applications.

References

1. OpenAI, “Generative AI and Large Language Models,” Available: <https://openai.com>
2. Flask Documentation, “Flask Web Framework,” Available: <https://flask.palletsprojects.com>
3. LangChain Documentation, “LangChain Framework,” Available: <https://www.langchain.com>
4. Python Software Foundation, “Python Documentation,” Available: <https://docs.python.org>
5. Bootstrap Documentation, “Bootstrap Framework,” Available: <https://getbootstrap.com>
6. ReportLab Documentation, “PDF Generation in Python,” Available: <https://www.reportlab.com/docs/>
7. Pressman, R. S., *Software Engineering: A Practitioner’s Approach*, McGraw-Hill
8. Sommerville, I., *Software Engineering*, Pearson